
Topic 4 – Async Research Assistant

Three sources, queried in parallel, answered with citations

team-M503-SWE-06

Sanan Garibli ■ Humbat Camalov ■ Farah Nematzada

Final Project Defense – September 2026

github.com/team-M503-SWE-06/research-assistant

The problem in one sentence

What the system does

A user asks a research question from the command line; we query Wikipedia, arXiv and a web-search API *concurrently*, bounded by a per-source timeout, and return one LLM-synthesised paragraph whose every claim carries a numbered citation — even when one of the three sources is down.

Inputs

- ▶ Question string (trimmed, non-empty, ≤ 500 chars)
- ▶ `--sources wiki,arxiv,web` subset
- ▶ `--no-cache` to bypass the source cache

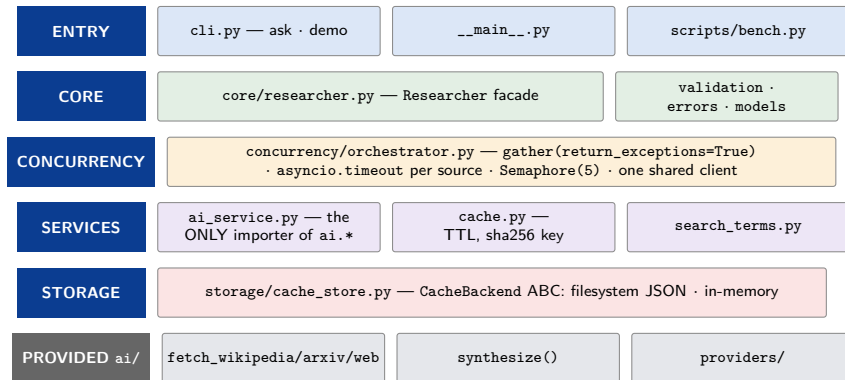
3 sources in parallel · **13** modules, ~ 620 lines · **90** offline tests · **95 %** statement coverage

Outputs

- ▶ `ResearchSession`: answer with citations + one `SourceOutcome` per source
- ▶ Numbered references with URLs, per-source timing line
- ▶ Exit 0 (possibly degraded, with a note) or 1 with `Error: ...`

Architecture

One rule drove everything: exactly one module imports `ai.*`.



Dependencies point downward only. `bootstrap.build_researcher()` is the single composition root — the CLI and the benchmark get an identically wired object graph.

Three design decisions we can defend

1. **One choke point for `ai.*`.** AIService is the only module that imports the provided package. Makes “never call provider SDKs from business logic” checkable by grep, and gives retries, timeouts and logging exactly one home.
2. **A CacheBackend ABC, composed — not inherited — into SourceCache.** Two implementations that share no code: one JSON file per key, or a dict for tests. TTL policy is orthogonal to storage, so composing avoids a class per (policy $\times$ storage) pair. Swapping in PostgreSQL is one subclass and one line in bootstrap.
3. **A long-lived semaphore and HTTP client, owned by the orchestrator.** A per-call semaphore bounds nothing once several questions run at once. Building the SSL trust store costs ~ 0.35 s and was being paid per client — hoisting it makes every later client nearly free.

Concurrency: how it works, and what it buys

```
tasks = [self._run_one(o, question, use_cache, self._client) for o in
          requested]
results = await asyncio.gather(*tasks, return_exceptions=True)

async with self._semaphore:                                # global bound
    async with asyncio.timeout(per_source_timeout):         # per source
        sources = await fetch(question, ..., client=client)
```

`return_exceptions=True` — one dead source does not cancel its siblings. **timeout per task** — a slow Wikipedia burns its own budget, not arXiv's. `Semaphore(5)` — bounds total in-flight calls, not per request.

Workload	N	Sequential	Concurrent	Speedup
5 sample research questions	5	31.4s	8.8s	3.6×

Captured 18 Sep 2026 · Windows 11, Intel 16-core, Python 3.13.14 · claude-sonnet-5 + Tavily · cache bypassed in both phases · raw log committed at [artefacts/bench.md](#).

Robustness: one concrete failure mode

Partial source failure — the web search dies, the answer still ships.

- ▶ **Injected.** A deliberately invalid Tavily key, no code changed. Also unit-tested by monkeypatching the arXiv fetcher to raise `ProviderError` on every attempt.
- ▶ **What happened.** `gather(return_exceptions=True)` returned the exception instead of cancelling siblings; the orchestrator recorded `SourceOutcome(source_count=0, error=...)`; the facade saw a non-empty source list and synthesised anyway.
- ▶ **What the user saw.** A complete cited answer, then one line:
Note: web unavailable (Tavily search failed: Client error '401 Unauthorized')
- ▶ **The complement is tested too.** When every source fails there is nothing honest to synthesise, so the facade raises `NoSourcesAvailableError` and the CLI exits 1 with a one-line message — never a traceback.

Evidence: `artefacts/degraded-run.txt`, committed from a live run.

Demo

```
$ python -m researcher ask "How does CRISPR-Cas9 gene editing work?"
```

```
A: CRISPR-Cas9 uses a guide RNA to direct the Cas9 nuclease to a matching  
DNA sequence, where it introduces a double-strand break [1][2]...
```

```
References:
```

- [1] (wikipedia) CRISPR gene editing
- [2] (arxiv) Structural basis of Cas9 target recognition
- [3] (web) How CRISPR works

```
(timing: wikipedia=0.37s, arxiv=0.38s, web=0.48s; * = cache hit)
```

Fallbacks: `--sources wiki,arxiv` if the search key is rate-limited · demo runs all five sample questions · a second run inside the TTL is served from cache and is visibly faster · `docker run --env-file .env research-assistant`.

Testing

90	95 %	16/16	60 %
tests, all offline	statement coverage	provided smoke tests	CI coverage gate

- ▶ **We mock at exactly one seam** — the `ai.*` functions. Everything below is the real object graph: a real orchestrator, a real cache, a real retry policy. Mocking the orchestrator would have meant testing our mocks instead of our wiring.
- ▶ **The concurrency tests** pin what the design promises: one task raising does not cancel the others; a slow source trips its own timeout; the semaphore's observed peak never exceeds its bound; a cache hit is served *before* a slot is taken.
- ▶ **The test that earned its keep:** printing an answer to a redirected stream on Windows raised `UnicodeEncodeError` mid-answer, after the user had already seen half of it. `stdout` is now reconfigured to UTF-8, and both that and the un-reconfigurable case are pinned.

Limitations & what we would do next

- ▶ **The search-term fallback is a heuristic, not a fix.** A stopwords list and a sliding window tuned against five questions. A question whose topic word is also a stopwords still does badly — the real fix is an endpoint the contract will not let us call.
- ▶ **Cache keys ignore `max_results`.** Ask for 3 then for 10 and you silently get 3. Not reachable from the CLI today, so it cannot bite yet — but it is a latent bug.
- ▶ **The concurrency bound is global, not per-host.** A slow arXiv can hold slots Wikipedia could have used.
- ▶ **Nothing is persisted beyond the cache.** A `ResearchSession` is built, printed and dropped.
- ▶ **With another week:** per-host rate limiting → persist sessions → replace the keyword heuristic. In that order, because that is the order in which a user notices.

Team contributions

Member	Pull requests	Share
Sanan Garibli	#1 config & models · #4 orchestrator · #7 tests · #10 Docker & CI · #17 final	5/17
Humbat Camalov	#3 AIService · #5 CLI & benchmark · #9 tests · #12 artefacts · #14 pitfall fixes · #16 slides	6/17
Farah Nematzada	#2 cache & storage · #6 tests · #8 tests · #11 README · #13 benchmark · #15 report	6/17

How we worked

- ▶ Feature branches named <member>/<short-description>; main is protected.
- ▶ Every PR used the template: what changed, why, how it was tested, what it does *not* do.
- ▶ The reviewer approves; the author merges. CI — ruff, mypy, pytest with the coverage gate, and a Docker build — must be green first.

Thank you

Questions?

`github.com/team-M503-SWE-06/research-assistant`

Sanan Garibli ■ Humbat Camalov ■ Farah Nematzada